22/8 Gestión de Configuración de Software (SCM)

1. Introducción a la Gestión de Configuración de Software (SCM)

La Ingeniería y Calidad de Software, a menudo, nos lleva a pensar más allá de las operaciones básicas de *commit* y *update* en el desarrollo de software. El **Software Configuration Management (SCM)** es una disciplina DE SOPORTE CON ENFOQUE PARAGUAS-TRANSVERSAL fundamental que aborda la complejidad inherente al desarrollo y mantenimiento de sistemas de software, asegurando su integridad y control a lo largo de todo su ciclo de vida.

1.1. El Software en Contexto

El software no es solo un conjunto de programas. Es un sistema complejo que abarca:

- **Programas:** El código ejecutable en sí.
- **Procedimientos:** Las instrucciones para usar o interactuar con el software.
- **Reglas:** Las normativas que rigen su funcionamiento y uso.
- **Documentación:** Manuales de usuario, especificaciones de diseño, requisitos, planes, etc..
- **Datos:** La información que el software procesa y almacena.

4P: personas, proceso (se adapta, automatizado con herramientas), producto, proyecto

- ▼ Personas

  Los mas importantes ya que de ellos surge todo lo demas.

- ▼ Procesos

  Procesos empíricos

  El equipo va a tener autonomía para decidir que proceso quiere usar y lo va a poder cambiar cuando le haga falta. Tienen que trabajar con ciclos de trabajo cortos, y si detectan oportunidades de mejora, adaptan. Voy ganando experiencia con los ciclos de vida iterativos cortos (máximo un mes).

  > La experiencia no se puede extrapolar, es específica de cada equipo, cliente, proyecto, etc.

  > **Repetibilidad** para obtener **previsibilidad**.

  Procesos definidos

  Está escrito, hay actividades definidas, roles, entradas, salidas, etc. Por ejemplo el proceso unificado. Se toma el proceso y se lo adapta al proyecto, sacando las cosas que no van a ser necesarias mediante guías de adaptación.

- ▼ Producto

  Disciplinas técnicas: requerimientos, análisis, diseño, implementación, prueba y despliegue.

  Dircetamente relacionadas con la creación de sw como producto.

- ▼ Proyecto

  Disciplinas de gestión: planificación y seguimiento y supervisión (o monitoreo y control). Verifica que lo que se planificó se cumple, mediante seguimiento/monitoreo. Y los hitos que salen de la

supervisión/control miden.

Seguimiento sin planificación no se puede.

Disciplinas protectoras (de soporte)

- SCM
- Aseguramiento de calidad
  - Proceso
  - Producto
- Métricas

El software, en esencia, es información estructurada con propiedades lógicas y funcionales. Se crea y mantiene en diversas formas y representaciones, evolucionando desde los requisitos iniciales hasta su estado final procesable por computadora.

1.2. La Evolución del Software y la Necesidad de SCM

El desarrollo de software es un proceso dinámico. Un proyecto de software atraviesa fases como la definición de **Requerimientos**, el **Diseño**, la implementación de **Fuentes** (código), y la generación de **Objetos** (compilados). Sin embargo, esta evolución no es lineal y **está sujeta a constantes cambios.**

Problemas Comunes sin SCM:

- **Pérdida de componentes o cambios:** Imagina un equipo trabajando en diferentes módulos, y sin un control adecuado, una versión esencial se pierde o se sobrescribe, resultando en un software que no arranca o funciona incorrectamente.
- **Desincronización fuente-objeto-ejecutable:** Tener código fuente que no coincide con el ejecutable liberado puede llevar a errores indetectables hasta la implementación, dificultando la depuración y mantenimiento.
- **Regresión de fallas:** Corregir un error en una parte del sistema e introducir, sin querer, un nuevo error en una funcionalidad que antes funcionaba, porque no se controló la propagación del cambio o no se realizaron pruebas exhaustivas de regresión.
- **Doble mantenimiento y superposición de cambios:** Dos desarrolladores trabajan en la misma sección de código simultáneamente y, al intentar integrar sus cambios, uno sobrescribe el trabajo del otro sin resolver los conflictos adecuadamente. Esto es un desperdicio de tiempo y esfuerzo.
- **Cambios no validados:** Implementar una nueva característica o un arreglo sin pasar por un proceso de revisión y prueba formal, lo que introduce inestabilidad y bugs en el sistema.

1.3. Origen de los Cambios en el Software

Los cambios son una constante en los proyectos de software y pueden surgir de diversas fuentes:

- **Cambios del negocio y nuevos requerimientos:** Las prioridades del mercado evolucionan, los usuarios solicitan nuevas funcionalidades, o la estrategia empresarial cambia. Por ejemplo, una empresa puede decidir añadir una función de inteligencia artificial a su plataforma existente.
- **Soporte de cambios de productos asociados:** El software puede interactuar con otros sistemas que también evolucionan, requiriendo adaptaciones. Un cambio en la API de un servicio externo integrado obligaría a modificar nuestro software.
- **Reorganización de prioridades de la empresa por crecimiento:** El crecimiento de una empresa puede llevar a la necesidad de nuevas herramientas o adaptar las existentes para escalar. Una funcionalidad diseñada para 100 usuarios puede necesitar una reingeniería para 1,000,000 de usuarios.

- **Cambios en el presupuesto:** La asignación de recursos puede variar, llevando a la priorización o descarte de ciertas funcionalidades o a la necesidad de optimizar costos en el desarrollo.
- **Defectos encontrados a corregir:** Los errores o bugs son inevitables y su corrección requiere modificaciones en el código o la documentación. Un reporte de un fallo crítico en producción demanda una acción inmediata.
- **Oportunidades de mejora:** Refactorizaciones de código, optimizaciones de rendimiento, o mejoras en la experiencia de usuario que no son estrictamente nuevos requerimientos, sino evoluciones del producto.

Ante esta realidad de cambio constante, el software se convierte en un "blanco móvil". La SCM surge como la disciplina esencial para gestionar esta complejidad y asegurar la **integridad del producto** a lo largo de su ciclo de vida.

2. SCM como Disciplina de Soporte

La SCM no es una actividad aislada, sino una **actividad "paraguas" o transversal** que acompaña a todo el proyecto de software, siendo relevante desde su inicio hasta el fin de su ciclo de vida. Forma parte de las **disciplinas de soporte del software**:

- **Aseguramiento de Calidad de Software (SQA):** Se enfoca en garantizar que se sigan los procesos correctos para producir software de alta calidad.
- **Prueba de Software:** Se dedica a verificar y validar el software para identificar defectos.
- **Control de Calidad de Proceso y Producto:** Monitoriza y evalúa la adherencia a los estándares y la calidad de los entregables.
- **Administración de Configuración de Software (SCM):** Proporciona la estructura para controlar los cambios y mantener la integridad de todos los artefactos del software.

Historia: El origen de la CM se remonta a mediados de la década de 1950, aplicándose inicialmente al desarrollo de hardware y control de producción. Posteriormente, sus principios fueron adoptados y adaptados para el desarrollo de software.

3. Definición y Propósito de SCM

La definición de SCM es crucial para entender su alcance:
SCM es una disciplina que aplica dirección y monitoreo administrativo y técnico para:

1. **Identificar y documentar** las características funcionales y técnicas de los **ítems de configuración**.
2. **Controlar los cambios** de esas características.
3. **Registrar y reportar** los cambios y su estado de implementación.
4. **Verificar la correspondencia** con los requerimientos.
   Esta definición es consistente con el estándar **ANSI/IEEE 828, 1990**.

3.1. ¿Por qué deberíamos gestionar la configuración del software?

El propósito principal de SCM es **establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida**. Esto se logra mediante:

- **Identificar** la configuración en un momento dado.
- **Controlar sistemáticamente** sus cambios.
- **Mantener su integridad y origen (trazabilidad)**.

La **integridad del producto** se mide por varios criterios:

- Satisface las necesidades del usuario.
- Puede ser fácil y completamente rastreado durante su ciclo de vida.
- Satisface criterios de *performance*.
- Cumple con sus expectativas de costo.

Sin SCM, un proyecto está "plagado de caos, errores, daños permanentes, baja productividad y una evolución del software inmanejable". SCM es el "mecanismo básico de control del proyecto que establece y mantiene la integridad de los productos de software a lo largo del ciclo de vida del proyecto".

4. Conceptos Clave para la Gestión de Configuración de Software

La SCM se construye sobre una serie de conceptos fundamentales:

4.1. Ítem de Configuración (IC) / Software Configuration Item (SCI)

▼ Ejemplos de items de config.

❖Plan de CM
❖ Propuestas de Cambio
❖ Visión
❖ Riesgos
❖ Plan de desarrollo
❖ Prototipo de Interfaz
❖ Guía de Estilo de IHM
❖ Manual de Usuario
❖ Requerimientos
❖ Plan de Calidad
❖ Arquitectura del Software
❖ Plan de Integración
❖ Planes de Iteración
❖ Estándares de codificación
❖ Casos de prueba
❖ Código fuente
❖ Gráficos, iconos, ...
❖ Instructivo de ensamble
❖ Programa de instalación
❖ Documento de despliegue
❖ Lista de Control de entrega
❖ Formulario de aceptación
❖ Registro del proyecto

Un **Ítem de Configuración (IC)** es cualquier artefacto que forma parte del producto o del proyecto, que puede sufrir cambios o necesita ser compartido entre los miembros del equipo, y sobre el cual necesitamos conocer su estado y evolución. En el contexto del software, se les llama **Software Configuration Items (SCIs)**.

Ejemplos de SCIs:

- **Documentación de Ingeniería:** Plan de SCM, propuestas de cambio, visión del producto, riesgos, plan de desarrollo, manual de usuario, requerimientos, plan de calidad, arquitectura del software, planes de iteración, estándares de codificación, casos de prueba.
- **Código y Binarios:** Código fuente, código ejecutable.
- **Artefactos de Interfaz y Diseño:** Prototipos de interfaz, guía de estilo de IHM, gráficos, íconos.

- **Artefactos de Despliegue y Proyecto:** Instructivos de ensamble, programas de instalación, documentos de despliegue, listas de control de entrega, formularios de aceptación, registro del proyecto.

El "Little Book of Configuration Management" amplía esta lista con más ejemplos de artefactos y productos de trabajo:

- **Planes:** Plan de Gestión de Ingeniería de Sistemas (SEMP), Plan de Desarrollo de Software (SDP), Manual de Estándares y Procedimientos de Software, Plan de Gestión de Configuración de Software (SCMP).
- **Especificaciones:** Especificación de Requisitos del Sistema (SRS), Especificación de Requisitos de Interfaz (IRS), Especificación de Diseño (descripciones de datos, arquitectura, módulos, interfaces, objetos).
- **Código Fuente:** Archivos de código, scripts de construcción (*build scripts*), listados de código, programas ejecutables, módulos vinculados, código legado (COTS, GFI, NDI).
- **Manuales:** Manuales de Operación e Instalación.
- **Especificaciones de Prueba:** Plan y procedimiento de prueba, casos de prueba y resultados registrados.
- **Descripción de Base de Datos:** Esquema y estructura de archivos, contenido inicial.
- **Documentos de Versión:** Documento de Descripción de Software/Versión (SVD) que incluye instrucciones de compilación y el entorno (nombre y versión del compilador, sistema operativo, etc.).
- **Productos COTS (Commercial Off-The-Shelf):** Compiladores, productos CASE (Computer-Aided Software Engineering), modelos, software de simulación.

Un SCI es un componente del sistema o software que tiene una relación jerárquica con otros SCIs.

4.2. Versión y Variantes

- **Versión:** Se define como la **forma particular de un artefacto en un instante o contexto dado**. El control de versiones se refiere a la evolución de un único IC o de cada IC por separado. Esta evolución se puede representar gráficamente como un grafo.
  - **Ejemplo:** *login.c* puede tener la versión 1.0, luego la versión 1.1 (con un bug corregido), y después la versión 1.2 (con una nueva característica).
- **Variante:** Es una versión de un IC (o de la configuración completa) que **evoluciona por separado**, representando configuraciones alternativas. Un producto de software puede adoptar distintas formas según el entorno donde se instale.
  - **Ejemplo:** Un software de gestión puede tener una variante para Windows y otra para Linux, o una variante con funcionalidades *premium* y otra *básica*. Esto podría representarse gráficamente como una bifurcación de una versión principal.

4.3. Configuración del Software

La **Configuración del Software** es el **conjunto de ítems de configuración con su correspondiente versión en un momento determinado**. Es, en esencia, una "instantánea" del sistema en un punto específico del tiempo, que puede ser reconstruida y replicada.

4.4. Repositorio

El **Repositorio** es un almacén de información que contiene todos los ítems de configuración.

- **Funcionalidad:** Mantiene la historia completa de cada IC, incluyendo sus atributos y las relaciones con otros ICs. Es fundamental para evaluar el impacto de los cambios propuestos antes de su

implementación. Puede consistir en una o varias bases de datos.

- **Funcionamiento:**
  - **Extracción (*Check-out*):** Los desarrolladores obtienen copias de los ICs del repositorio para trabajar en ellos.
  - **Devolución (*Check-in*):** Una vez que los cambios se realizan y validan en el entorno de trabajo individual, se devuelven al repositorio.
  - **Construcción (*Build*):** El repositorio de fuentes se utiliza para construir los objetos y ejecutables, transfiriendo y actualizando componentes.

Tipos de Repositorios:

- **Centralizados:** Un servidor único almacena todos los archivos y sus versiones. Ofrecen mayor control a los administradores, pero son un punto único de falla; si el servidor cae, "estamos al horno".
  - **Ejemplo:** CVS, SVN (aunque SVN puede simular algunos aspectos distribuidos).
- **Descentralizados:** Cada cliente o desarrollador posee una copia completa del repositorio. Esto proporciona mayor redundancia (si un servidor falla, otra copia puede restaurarlo) y habilita *workflows* más flexibles.
  - **Ejemplo:** Git, Mercurial.

4.5. Línea Base (*Baseline*)

Una **Línea Base** es una **configuración que ha sido revisada formalmente y sobre la que se ha llegado a un acuerdo**. Sirve como base para desarrollos posteriores y solo puede cambiarse a través de un procedimiento formal de control de cambios.

- **Identificación:** Se utilizan etiquetas para "marcar" las *baselines*, pero no deben confundirse con la versión del producto final.
- **Propósito:** Permiten "ir atrás en el tiempo" y reproducir el entorno de desarrollo en un momento dado del proyecto. Esto es vital para depurar problemas antiguos, realizar entregas a clientes con versiones específicas, o cumplir con requisitos contractuales.
- **Componentes:** Pueden ser de especificación (requisitos, diseño) o productos que han pasado por un control de calidad definido.
- **Ejemplo:** Después de la fase de diseño, el equipo de desarrollo y los *stakeholders* acuerdan y "congelan" el documento de arquitectura como la "Línea Base de Diseño". Cualquier cambio futuro a la arquitectura debe seguir un proceso formal.
- El "Little Book" detalla cómo se establecen las baselines en los hitos del proyecto:
  - **Functional Baseline:** Después de la Revisión de Requisitos del Sistema (SRR), con las especificaciones del sistema y las asignaciones de presupuesto de rendimiento.
  - **Allocated Baseline:** Después de la Revisión de Diseño del Sistema (SDR) o Revisión Preliminar de Diseño (PDR), con las especificaciones de requisitos de software (SRS) e interfaz (IRS).
  - **Preliminary Product Baseline / Detail Design Baseline:** Después de la Revisión Crítica de Diseño (CDR), con el diseño detallado y los procedimientos de prueba.
  - **System Performance Test Baseline:** Después de la Revisión de Disponibilidad de Pruebas (TRR) / Prueba de Aceptación en Sitio, con la aprobación del diseño detallado para iniciar pruebas y el software e equipo de prueba con baseline.
  - **Product Baseline:** Después de la Auditoría de Configuración, verificando todos los ICs contra la documentación del sistema.

4.6. Ramas (*Branches*)

Las **Ramas** permiten bifurcar el desarrollo de un proyecto.

- **Creación:** Existe una rama principal (a menudo llamada *trunk* o *master*) de la que se derivan las ramas. Las ramas se crean por diversas razones, como el desarrollo de nuevas características, correcciones de errores, o experimentación sin afectar la rama principal.
- **Propósito:** Permiten la experimentación y el desarrollo paralelo. Una rama puede ser descartada si la experimentación no tiene éxito o, más comúnmente, **integrada** de nuevo en la rama principal.
- **Integración (*Merge*):** La operación de integración de ramas se llama *merge*. Transfiere los cambios de una rama a otra (típicamente a la principal). Durante el *merge*, pueden surgir conflictos si las mismas partes del código han sido modificadas de manera diferente en ramas distintas, requiriendo una **resolución de conflictos** (a menudo con herramientas *diff*). Idealmente, todas las ramas deberían eventualmente integrarse a la principal o ser descartadas.

5. Elementos Principales y Actividades Fundamentales de SCM

Los elementos principales de SCM se agrupan en cuatro actividades fundamentales:

5.1. Identificación de Ítems de Configuración (CI Identification)

Esta actividad es la base de la SCM. Implica:

- **Identificación unívoca:** Asignar una identificación única a cada ítem de configuración.
- **Convenciones y reglas de nombrado:** Establecer estándares claros para nombrar los archivos, módulos, documentos, etc., para asegurar consistencia y fácil comprensión.
- **Definición de la estructura del repositorio:** Organizar el repositorio de manera lógica para almacenar los ICs.
- **Ubicación dentro de la estructura:** Definir dónde se guardará cada tipo de IC para facilitar su acceso y gestión.

Ejemplo Práctico: En un proyecto de desarrollo web, los ICs podrían ser:

- **Producto:** Especificación de Requerimientos de Software (ERS), Arquitectura, Código fuente del *backend*, Manual de Usuario.
- **Proyecto:** Plan de Proyecto, Cronograma.
- **Iteración (Sprint):** Plan de Iteración, Reporte de Defectos del Sprint actual.

Formato para los trabajos (después del ítem de producto se estructura el repositorio):

Ítem de producto:

`Autogestion_<<tema>>.<<ext>>` → `ejemplo: Autogestion_ERS.docx`

Ítem de proyecto:

`PorFin_PlanProyecto.docx`

Ítem de iteración:

`PorFin_PlanIteracion<<nro>>.docx`

5.2. Control de Cambios (Change Control)

Esta es quizás la actividad más visible de SCM, enfocada en gestionar las modificaciones a los ICs. Es un procedimiento formal que involucra a diferentes actores y una evaluación rigurosa del impacto del cambio.

Proceso de Control de Cambios:

1. **Propuesta de Cambio (Requerimiento de Cambio):** Se origina cuando se identifica la necesidad de modificar uno o varios ítems de configuración que ya están bajo una línea base.
2. **Análisis de Impacto:** Se evalúan aspectos técnicos, efectos colaterales, costos, impacto general en el proyecto, limitaciones, criterios de auditoría y revisión, y otras tareas.
3. **Revisión y Decisión:** Un **Comité de Control de Cambios (CCC o CCB)** o una **Junta de Revisión de Ingeniería (ERB)** revisa la propuesta. Este comité toma la decisión de aceptar o rechazar el cambio.
   - **Comité de Control de Cambios (CCC/CCB):** Está compuesto por representantes de todas las áreas involucradas: Análisis, Diseño, Implementación, Testing y otros interesados. Su objetivo es asegurar que los cambios propuestos sean evaluados sistemáticamente, que solo se aprueben los necesarios y que se implementen los aprobados, clasificando correctamente los cambios y determinando la urgencia.
   - **Posibles Acciones del CCC/ERB:** Rechazar, asignar para estudio, asignar para corrección, adjuntar a un reporte de problema existente, reanalizar, posponer análisis, diferir, identificar requisitos de prueba, construir y liberar sistema, o cerrar el reporte.
4. **Aceptación o Rechazo:** La decisión del CCC/CCB se comunica. Si se acepta, se genera una Orden de Cambios de Ingeniería y se ubica en la cola de cambios.
5. **Notificación a las Partes:** Se informa a todos los *stakeholders* afectados sobre la decisión.
6. **Implementación y Desarrollo:** Una vez aprobado, el cambio se implementa por el equipo de desarrollo.

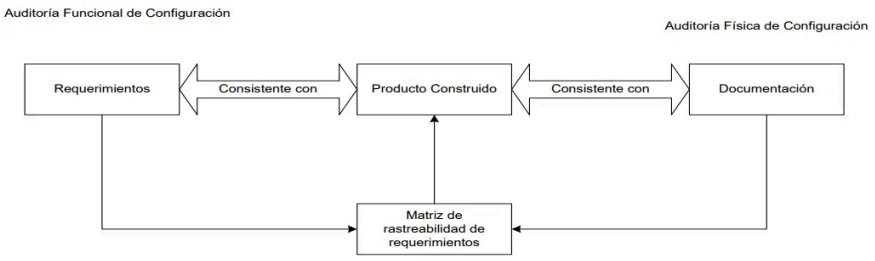
El "Little Book" enfatiza la importancia de que el proceso de gestión de cambios sea **simple, consistente con la cultura del proyecto y apoyado por las herramientas**. También establece que **CM nunca debe ser ignorado** por razones de presupuesto o cronograma.

Reglas sobre el Control de Cambios (Little Book):

- **Regla 3:** El proceso de gestión de cambios debe ser simple, consistente con la cultura y los requisitos del proyecto, y respaldado por los métodos y herramientas del proyecto.
- **Regla 4:** Las acciones, roles y responsabilidades de la Junta de Revisión de Ingeniería (ERB) y la Junta de Control de Cambios (CCB) deben documentarse.
- **Regla 5:** El control de cambios debe ser un foco principal del programa y debe integrarse en la cultura del proyecto.
- **Regla 7:** Todas las propuestas de cambios a productos controlados por CM deben clasificarse adecuadamente.
  - **Cambios Clase I:** Afectan funcionalidad, rendimiento, costo, cronograma, garantías o acuerdos contractuales.
  - **Cambios Clase II:** Cambios menores de documentación que no afectan factores de Clase I.
  - **Cambios Externos:** Afectan información de propiedad o gestión conjunta sin relación contractual (ej. interfaces).

- **Cambios Internos:** Afectan información de propiedad o uso de otros dentro de la organización, pero no aprobados o entregados al adquirente.

5.3. Auditorías de Configuración (Configuration Audits)



Las auditorías son evaluaciones formales para asegurar la integridad de los ítems de configuración. Sirven a los procesos de **validación y verificación**.

- **Auditoría Física de Configuración (PCA - Physical Configuration Audit):** Asegura que lo que se ha indicado para cada SCI en la línea base o en una actualización se ha alcanzado realmente. Verifica la correspondencia entre el "producto construido" y la "documentación".
  - **Ejemplo:** Verificar que todos los módulos de código listados en el SVD (Software Version Description) de la versión 2.0 existen en el repositorio con las etiquetas correctas y que la documentación de instalación corresponde con el producto entregado.
- **Auditoría Funcional de Configuración (FCA - Functional Configuration Audit):** Evaluación independiente que controla que la funcionalidad y el rendimiento reales de cada ítem de configuración sean consistentes con la especificación de requerimientos. Asegura que el "producto construido" es "consistente con los requerimientos".
  - **Ejemplo:** Realizar pruebas exhaustivas para confirmar que todas las características descritas en el documento de requerimientos funcionan como se espera y que el sistema cumple con los criterios de rendimiento especificados.

Vínculo con V&V:

- **Validación:** Asegura que "el problema es resuelto de manera apropiada, que el usuario obtenga el producto correcto".
- **Verificación:** Asegura que "un producto cumple con los objetivos preestablecidos, definidos en la documentación de líneas base".

5.4. Informes de Estado (Status Reporting)

Esta actividad se encarga de **mantener los registros de la evolución del sistema**. Gestiona una gran cantidad de información y, por lo tanto, suele implementarse mediante procesos automáticos. Incluye reportes de trazabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida.

Preguntas clave que puede responder un sistema de informes de estado:

- ¿Cuál es el estado actual de un ítem de configuración específico?.
- ¿Un requerimiento de cambio ha sido aprobado o rechazado por el CCB?.
- ¿Qué versión de un ítem de configuración implementa un requerimiento de cambio aprobado? (Por ejemplo, saber qué componente contiene una mejora específica).
- ¿Cuál es la diferencia exacta entre una versión y otra dada de un artefacto?.

El "Little Book" denomina a esta actividad **Status Accounting** y la describe como la "grabación y reporte formalizados de los documentos de configuración establecidos, el estado de los cambios propuestos y el estado de la implementación de los cambios aprobados". Proporciona un registro "accesible y actual del estado de cada pieza de información controlada".

6. Plan de Gestión de Configuración (SCM Plan)

La SCM, como cualquier disciplina crítica, **también se planifica**. Un Plan de Gestión de Configuración (SCMP) es un documento que detalla cómo se aplicará la SCM a un proyecto.

¿Qué debería incluir un SCMP?

- **Introducción:** Propósito, alcance y aplicabilidad del plan.
- **Organización:** Relación e integración de SCM con todas las organizaciones del proyecto (Calidad, Ingeniería, Cliente, Gestión, etc.).
- **Fases y Hitos del Proyecto:** Describe la evolución del proyecto respecto a SCM y el establecimiento de líneas base de desarrollo y formales.
- **Identificación de Configuración:** Selección de artefactos, identificadores de configuración, establecimiento de bibliotecas del proyecto y convenciones de nombres/numeración.
- **Gestión de Interfaces:** Procedimientos para la identificación de requisitos de interfaz, establecimiento de acuerdos de interfaz internos y externos, y participación en grupos de trabajo de control de interfaz.
- **Status Accounting:** Cómo y cuándo se registrará y liberará la información de estado para el proyecto, incluyendo la versión actual de los productos y el estado de los cambios.
- **Auditorías de Configuración:** Planes y procedimientos para realizar auditorías y verificación de un producto.
- **Gestión de Subcontratistas:** Proceso y procedimientos para asegurar el cumplimiento de los subcontratistas con los requisitos del proyecto.
- **Reglas de nombrado** de los ítems de configuración.
- **Herramientas** a utilizar para SCM.
- **Roles e integrantes** del Comité de Control de Cambios.
- **Procedimiento formal de cambios**.
- **Plantillas de formularios**.
- **Procesos de Auditoría**.

Mejores Prácticas de SCM (Little Book):

- **BP #1: Hacer de la CM trabajo de todos:** Crear un entorno donde la adhesión al proceso de CM establecido sea la única forma aceptable de hacer negocios.
- **BP #2: Crear un entorno y proceso de ingeniería que habilite la CM:** Definir el proceso de ingeniería/CM primero y refinarlo, buscando áreas que causen dificultad a desarrolladores y *testers* en el mantenimiento de CM y realizando cambios.
- **BP #3: Definir y documentar el proceso CM/Ingeniería, luego seleccionar las herramientas:** Las herramientas deben apoyar el proceso, no al revés. El *output* de las herramientas de CM debe ser usable directamente.
- **BP #4: El personal de CM debe tener experiencia técnica:** Para desarrollar e implementar procesos y procedimientos sólidos.

- **BP #5: El Plan y Procedimientos de CM deben desarrollarse al inicio del proyecto:** Como cualquier plan de desarrollo de software, planificando personal, presupuesto, cronograma, procesos y procedimientos.

7. SCM en Ambientes Ágiles

Los ambientes ágiles, que enfatizan la respuesta al cambio sobre seguir un plan rígido, requieren una adaptación de la SCM tradicional. El **Manifiesto Ágil** prioriza "individuos e interacciones sobre procesos y herramientas", "software funcionando sobre documentación exhaustiva", "colaboración con el cliente sobre negociación contractual" y "respuesta al cambio sobre seguir un plan".

7.1. Adaptación de SCM en Agile

En un entorno ágil, la SCM se redefine para ser más ligera, flexible y orientada al equipo.

- **Sirve a los practicantes:** La SCM debe apoyar al equipo de desarrollo, no imponer un control excesivo.
- **Seguimiento y coordinación:** En lugar de controlar a los desarrolladores, la SCM debe rastrear y coordinar el desarrollo.
- **Responde a los cambios:** En lugar de tratar de evitarlos, la SCM ágil abraza los cambios y proporciona mecanismos para gestionarlos eficientemente.
- **Transparencia y automatización:** Busca ser "sin fricción", automatizando tanto como sea posible para reducir la carga administrativa y acelerar los procesos.
- **Coordinación y automatización frecuente y rápida:** Es esencial para la integración continua y la entrega frecuente.
- **Eliminar el desperdicio:** No agregar nada que no aporte valor, enfocándose en la eficiencia.
- **Documentación *Lean* y Trazabilidad:** Prioriza la documentación mínima necesaria y la trazabilidad de los cambios. Los scripts de construcción bien documentados son una forma de "documentación ejecutables".
- **Feedback continuo y visible:** Sobre calidad, estabilidad e integridad del sistema.

Consejos para SCM en Agile:

- **Responsabilidad de todo el equipo:** La SCM no es solo tarea de un rol específico, sino una responsabilidad compartida.
- **Automatizar lo más posible:** Herramientas de integración continua, pruebas automatizadas y despliegue continuo son fundamentales.
- **Educar al equipo:** Capacitar a todos los miembros sobre las prácticas y herramientas de SCM.
- **Tareas de SCM embebidas:** Las actividades de SCM se integran en las tareas diarias del *sprint*, no se realizan como un paso separado al final.

7.2. Gestión de Construcciones (*Build Management*) en Agile SCM

La **gestión de construcciones** es fundamental en SCM ágil, ya que la integración de componentes es clave para que una aplicación sea útil. Una *build* es el paso del proceso donde se integra toda o parte de una aplicación, incluyendo la recuperación de componentes, compilación, generación de archivos fuente a partir de metadatos, etc., hasta obtener un componente de valor.

Tipos de Construcciones (*Builds*) en Agile:
Se identifican tres patrones principales de *builds*, cada uno con diferente alcance, frecuencia y objetivo:

1. **Private System Build (Construcción de Sistema Privado):**

- **Consumidor:** El desarrollador individual que trabaja en una tarea aislada.
- **Propósito:** Verificar la consistencia y corrección de sus cambios **antes de *commitirlos*** a la línea de código principal. Aísla al equipo de problemas que podrían impedir el progreso de otros.
- **Frecuencia:** Una o más veces por tarea de desarrollo.
- **Pasos Típicos:**
  - **Actualización del Workspace (*Workspace Update*):** Asegurarse de que los cambios del desarrollador incorporen los cambios más recientes ya integrados exitosamente en la línea de código por otros. Esto puede implicar fusionar conflictos.
  - **Construcción y Ejecución de Pruebas:** Realizar una *build* (generalmente incremental, pero a veces completa) y ejecutar pruebas para asegurar que los componentes se construyen y ejecutan correctamente.
- **Comparación:** Este tipo de *build* prioriza la velocidad, pudiendo ser incremental y con un riesgo aceptable de pequeños errores, ya que se realiza en un entorno aislado.

2. **Integration Build (Construcción de Integración):**
   - **Consumidor:** El equipo de desarrollo completo.
   - **Propósito:** Incorporar las tareas de desarrollo más recientemente completadas en un resultado único, estable, consistente y correcto. Proporciona seguridad adicional de que el código se integra en un entorno "limpio" y permite pruebas a nivel de integración. El resultado de una *Integration Build* exitosa se utiliza para actualizar los *workspaces* individuales.
   - **Frecuencia:** Preferiblemente después de cada tarea de cambio o varias veces al día (ej., diariamente o nocturna).
   - **Detalles:** Puede ser una *build* completa si el tiempo lo permite. Evita la "integración *big bang*" al final de una iteración, abordando los problemas de integración de forma temprana y agresiva.
   - **Comparación:** Las *Integration Builds* operan en las últimas versiones (*tips*) de la rama, ejecutan pruebas exhaustivas y su objetivo es la calidad de la línea de código y el *feedback* al equipo.
3. **Release Build (Construcción de Liberación):**
   - **Consumidor:** Equipo de pruebas (QA/V&V) y clientes del equipo de desarrollo.
   - **Propósito:** Empaquetar el software para su distribución e instalación.
   - **Frecuencia:** Al final de cada iteración/lanzamiento. Ocurren con mucha menos frecuencia que las *Integration Builds*.
   - **Pasos Típicos:**
     - Aplicar una etiqueta a la línea de código.
     - Obtener una copia de la línea de código en una máquina de *build* "dorada" (entorno controlado).
     - Construir la aplicación.
     - Posiblemente probar la aplicación y reportar los resultados.
     - Empaquetar los entregables (ej., en un archivo ZIP, o ejecutar un instalador).
     - Enviar un informe por correo electrónico con los resultados.
   - **Diferencias Clave con Integration Build:** Una *Release Build* típicamente aplica una etiqueta, empaqueta la versión para instalación y distribución, y puede incluir informes automatizados de CM (como listas de materiales o informes de solicitudes de cambio). No siempre opera en las últimas versiones, sino en la versión específica que se va a liberar.

Escala y Agilidad: Estos tipos de *builds* operan a diferentes niveles de escala en términos de visibilidad (quién ve el resultado) y frecuencia/duración (impacto en la planificación del proyecto). Es importante lograr un equilibrio entre velocidad y estabilidad. La disciplina y precisión requeridas aumentan desde el *Private System Build* hasta el *Release Build*, donde cualquier error pasado debe ser capturado. Es crucial establecer estos procesos de *build* y despliegue **temprano en el proyecto** para que evolucionen con el producto.

7.3. Construcción Continua (*Continuous Building*)

Está directamente relacionado con la **Integración Continua**. Al tomar en serio el proceso de *build* y despliegue a lo largo del desarrollo, se asegura un progreso constante del producto. Si una *build* falla, se obtiene una advertencia temprana de problemas en la línea de código. Construir temprano y a menudo también asegura la **reproducibilidad de la *build*** y la integridad de las líneas de código, facilitando la comunicación entre los *stakeholders*.

7.4. Optimización de las Construcciones y Valores Ágiles

Para construir agilidad en el proceso, las *builds* deben ser fáciles de hacer y repetibles, utilizando herramientas que se integren bien y limitando los procedimientos manuales.

- **Dependencias de la *Build*:** Es crucial gestionarlas y minimizarlas para mejorar los tiempos de recompilación y reducir conflictos.
- **Arquitectura de la *Build*:** La estructura del sistema de *build* debe soportar la reproducibilidad, la *testabilidad* de unidades pequeñas y *builds* más rápidas (o paralelismo).
- **Múltiples Tipos de *Build*, un Único Mecanismo de *Build*:** Es más fácil usar el mismo o similar mecanismo para iniciar los diferentes tipos de *builds*, con pequeños cambios en la invocación o el entorno para especificar el tipo.

Mejora del Rendimiento de la *Build*:

- Almacenar objetivos intermedios en el repositorio para mejorar los tiempos de *build* incrementales.
- Dividir los proyectos en módulos construibles independientemente.
- Invertir en hardware más potente (el ahorro de tiempo de los desarrolladores compensa el costo).
- Considerar diferentes utilidades de *build*, compiladores y *linkers* que puedan ofrecer mejores tiempos de respuesta.

Conexión con los Valores del Manifiesto Ágil:

- **Individuos e Interacciones sobre Procesos y Herramientas:** Las herramientas de SCM deben apoyar la forma de trabajar del equipo. Todos en el equipo de desarrollo deberían poder realizar, al menos, un *Private System Build*.
- **Software Funcionando sobre Documentación Exhaustiva:** La SCM puede automatizar políticas y procesos de desarrollo con conocimiento ejecutable (scripts de *build*) en lugar de depender de documentación manual que a menudo está desactualizada.
- **Colaboración con el Cliente sobre Negociación Contractual:** SCM facilita la comunicación y la interacción con los *stakeholders* al proporcionar *builds* frecuentes, lo que permite gestionar mejor las expectativas del cliente.
- **Respuesta al Cambio sobre Seguir un Plan:** La SCM se trata de facilitar el cambio, no de prevenirlo. Al construir de forma frecuente y adecuada, el equipo y los clientes tienen una mejor idea del estado actual y qué necesita ser cambiado.

7.5. Puntos de Debate en SCM Ágil

- **¿Qué pasa con el Comité de Control de Cambios (CCC/CCB)?** En entornos ágiles, la formalidad de un CCB tradicional para cada cambio menor puede ralentizar el desarrollo. A menudo, la responsabilidad de aprobar cambios se delega al equipo de desarrollo, con revisiones de código y pruebas automatizadas actuando como "puertas de calidad". Para cambios mayores, especialmente aquellos que impactan el negocio o el alcance del proyecto, podría seguir existiendo una junta de *stakeholders* o producto para tomar decisiones estratégicas.
- **¿Qué ítems de configuración podemos tener?** Los ICs siguen siendo relevantes, pero el enfoque ágil promueve una documentación "justo a tiempo" y "suficientemente buena". Los ICs podrían centrarse más en el código fuente, los scripts de prueba, los scripts de *build* y los documentos de requisitos de alto nivel, con menos énfasis en la documentación formal detallada de cada módulo.
- **¿Qué pasa con las auditorías?** Las auditorías físicas y funcionales siguen siendo importantes para la calidad y el cumplimiento. En ágil, se pueden realizar auditorías más ligeras y frecuentes, o integrarse en las actividades de *testing* continuo y revisiones de *sprint*. Por ejemplo, las *Definition of Done* de un *sprint* podrían incluir criterios de auditoría funcional y física automatizados.
- **¿Qué pasa con los reportes de estado?** Los informes de estado son vitales para la transparencia en ágil. Las herramientas de CI/CD y los paneles de control (*dashboards*) pueden proporcionar información en tiempo real sobre el estado de la *build*, los resultados de las pruebas y el progreso del desarrollo, reemplazando los informes manuales y formales por datos más dinámicos y accesibles.

8. Vínculos con Otras Unidades y Disciplinas

La SCM es inherentemente interdisciplinaria:

- **Gestión de Proyectos:** La SCM proporciona la visibilidad necesaria para la planificación y el control de proyectos, permitiendo pronosticar fechas de lanzamiento (reduciendo la "resbaladiza" de los cronogramas), evaluar el presupuesto y el cronograma en puntos discretos. Un Plan de SCM es parte integral del plan general del proyecto.
- **Aseguramiento y Control de Calidad de Software (SQA/SQC):** La SCM es una disciplina de soporte para la calidad. Las auditorías de configuración son cruciales para asegurar la calidad. La promoción de información a la jerarquía de CM debe basarse en la finalización exitosa de una "puerta de calidad".
- **Pruebas de Software:** El control de cambios asegura que los sistemas de software para pruebas tienen una configuración conocida y son reproducibles. Las *Integration Builds* ejecutan pruebas exhaustivas, y las auditorías funcionales verifican la correspondencia con los requerimientos mediante pruebas.
- **Despliegue (Deployment):** Las *Release Builds* son el paso final de SCM para empaquetar el software para su instalación y distribución, siendo el puente entre el desarrollo y el despliegue del software. La SCM asegura la integridad del producto que se despliega.
- **Trazabilidad:** La SCM permite la trazabilidad completa del software, desde los requisitos hasta el código y los cambios implementados, facilitando el mantenimiento y la comprensión del sistema. La trazabilidad es crítica para la validación y verificación.

Conclusión

La Gestión de Configuración de Software es una disciplina esencial, no solo para la organización y el control técnico, sino como la **fundación de cualquier proyecto de software exitoso**. Sin una SCM efectiva, incluso los equipos más talentosos con presupuestos robustos y procesos de desarrollo avanzados, pueden ver cómo la disciplina del proyecto colapsa, dejando el éxito al azar. Ya sea en enfoques tradicionales o ágiles, la clave reside en establecer un proceso de SCM que se adapte a la cultura y necesidades del proyecto, apoyado por herramientas adecuadas, y que se integre

profundamente en el *workflow* del equipo. Al dominar los conceptos y prácticas de SCM, no solo se gestiona el cambio, sino que se facilita la evolución controlada y exitosa de cualquier producto de software.

TP4 SCM EVALUABLE — Caso 2 Resuelto Liga Regional de futbol

Items de configuración

- Sprint backlog
- Planif sprint
- User Stories
- Arquitectura
- Product Backlog
- Metricas
- Codigo fuente

Tipos de ITEM

SPRINT

SPRINT

PRODUCTO

PRODUCTO

PRODUCTO

PRODUCTO

PRODUCTO